

Rapport d'activité - 3ème année

Bachelor Architecture des logiciels - ESGI Paris

Année scolaire : 2025-2026

Étudiant : AIDEL Massinissa

Entreprise d'accueil : SAS MOUSTAK / Matisse Food

Tuteur / maître d'apprentissage : BOUZEKRI Mihran, employé polyvalent

Type de contrat : contrat d'apprentissage / alternance

Dates du contrat : du 02/01/2026 au 09/11/2026

Période couverte par le rapport : janvier à mai 2026, soit environ cinq mois réalisés au moment du rendu

Lieu d'exécution : 10 bis rue Barbes, 94200 Ivry-sur-Seine

[Emplacement du logo de l'entreprise]

[Emplacement du logo de l'ESGI Paris]

Remerciements

Je souhaite tout d'abord remercier la SAS MOUSTAK, exploitant l'activité Matisse Food à Ivry-sur-Seine, pour m'avoir accueilli dans le cadre de mon alternance de troisième année en Bachelor Architecture des logiciels à l'ESGI Paris. Cette expérience m'a permis de travailler sur un projet concret, directement relié aux besoins d'un commerce de proximité, avec des contraintes d'usage réelles et un objectif clairement mesurable : améliorer l'expérience client et soutenir la visibilité numérique du restaurant.

Je remercie particulièrement Monsieur Mihran Bouzekri, mon maître d'apprentissage, pour sa disponibilité, ses retours et la confiance qu'il m'a accordée dans la conception et la réalisation de la solution Matisse Food. Son retour terrain a été important pour orienter les choix fonctionnels, notamment sur la simplicité du parcours client, l'utilisation par le personnel en caisse et la facilité de gestion des lots.

Je remercie également l'équipe pédagogique de l'ESGI Paris et du CFA ANAPIJ pour l'encadrement de cette année d'alternance, ainsi que pour les enseignements techniques qui m'ont aidé à structurer mon travail : développement frontend avec React, API Node.js, conteneurisation Docker, bases de données, qualité logicielle et méthodes agiles.

Enfin, je remercie les personnes qui ont pu tester l'application ou donner un avis sur l'ergonomie. Ces retours ont contribué à rendre le projet plus clair, plus accessible et plus adapté à un usage rapide sur smartphone.

Sommaire

1. Introduction

2. Partie 1 - Contexte de l'entreprise

2.1 Présentation de la SAS MOUSTAK / Matisse Food

2.2 Positionnement du service et rôle occupé

3. Partie 2 - Missions réalisées

3.1 Mission 1 : cadrage, UX et site client Matisse Food

3.2 Mission 2 : API, base de données et logique de jeu sécurisée

3.3 Mission 3 : interface d'administration et suivi opérationnel

3.4 Mission 4 : déploiement, conteneurisation et mise en production

3.5 Organisation, outils et méthode de travail

4. Partie 3 - Bilan et recul sur les missions

4.1 Regard critique et axes d'amélioration

4.2 Apports à l'entreprise

4.3 Apports personnels et projet professionnel

5. Conclusion

6. Annexes

Annexe 1 : fiche d'identité de l'entreprise

Annexe 2 : fiche projet Matisse Food

Annexe 3 : CV mis à jour à insérer par l'étudiant

Annexe 4 : clause de confidentialité signée à joindre si exigée par l'école

Introduction

Dans le cadre de ma troisième année de Bachelor Architecture des logiciels à l'ESGI Paris, j'ai effectué une alternance au sein de la SAS MOUSTAK, entreprise située à Ivry-sur-Seine et rattachée au secteur de la restauration rapide. Le contrat d'apprentissage a débuté le 2 janvier 2026 et doit se terminer le 9 novembre 2026. Le présent rapport couvre la période déjà réalisée au moment du rendu, soit environ cinq mois d'activité en entreprise.

L'objectif de ce rapport est de présenter le contexte de mon alternance, les missions techniques qui m'ont été confiées et le recul que je peux prendre sur cette expérience. Mon travail principal a porté sur la conception et le développement d'une application web nommée Matisse Food, accessible à l'adresse <https://matissefood.aimassi.dev>. Cette application propose aux clients du restaurant un parcours de jeu simple : récupérer un code unique en caisse, accéder au site depuis leur téléphone, soutenir le restaurant via un avis Google, puis faire tourner une roue permettant de gagner des récompenses. Une interface d'administration a également été développée à l'adresse <https://matissefood.aimassi.dev/admin> afin que le personnel puisse générer des codes, gérer les lots, suivre les statistiques et valider les récompenses en caisse.

Ce projet avait un intérêt particulier car il liait des problématiques très concrètes d'un commerce local avec des compétences attendues dans ma formation. Il ne s'agissait pas uniquement de produire une interface attractive, mais de concevoir une solution complète : expérience utilisateur, backend, modèle de données, sécurité, administration, déploiement et maintenance. Le projet a donc mobilisé des compétences frontend avec React, backend avec Node.js et Express, persistance avec PostgreSQL et Prisma, déploiement avec Docker, ainsi que des outils de suivi comme Notion, Trello et GitHub.

Le rapport est organisé en trois grandes parties. La première présente l'entreprise d'accueil et le cadre dans lequel j'ai travaillé. La deuxième détaille les missions réalisées, en allant de la conception du parcours client jusqu'au déploiement technique. La troisième propose un bilan critique : apports pour l'entreprise, compétences acquises, difficultés rencontrées et pistes d'amélioration.

Partie 1 - Contexte de l'entreprise

1.1 Présentation de la SAS MOUSTAK / Matisse Food

La SAS MOUSTAK est l'entreprise d'accueil dans laquelle j'ai effectué mon alternance. D'après les informations issues de la convention et du contrat d'apprentissage, l'établissement d'exécution est situé au 10 bis rue Barbes, 94200 Ivry-sur-Seine. Le numéro SIRET mentionne dans le contrat est 92124341600010 et le code APE est 5610C, correspondant à la restauration de type rapide. L'effectif total indiqué est de quatre salariés.

L'activité de Matisse Food s'inscrit dans un marché très concurrentiel : la restauration rapide de proximité. Ce type d'activité repose sur plusieurs leviers importants : la qualité des produits, la rapidité de service, la proximité géographique, la visibilité locale et la satisfaction client. Dans ce contexte, la présence numérique devient un facteur de différenciation. Un client potentiel consulte souvent les avis Google, les photos, la localisation et les horaires avant de se déplacer ou de commander. Pour un restaurant local, encourager les retours clients et rendre l'expérience plus engageante peut donc avoir un impact direct sur la réputation en ligne.

L'entreprise est de petite taille, ce qui modifie la manière de travailler par rapport à une grande organisation. Les circuits de décision sont courts, les retours sont rapides et les besoins sont directement exprimés par les personnes qui utilisent ou supervisent la solution. En revanche, cela implique aussi des contraintes fortes : le temps disponible du personnel est limité, les outils doivent être simples, et la solution ne doit pas ajouter de complexité opérationnelle. Une application trop lourde, trop technique ou difficile à administrer aurait peu de chances d'être utilisée durablement.

Le projet Matisse Food a donc été pensé comme un outil pratique au service du restaurant. Il devait répondre à trois enjeux principaux. Le premier était marketing : augmenter l'engagement client et inciter les clients satisfaits à laisser un avis Google. Le deuxième était opérationnel : permettre au restaurant de distribuer des codes et de valider les lots sans procédure complexe. Le troisième était technique : construire une application suffisamment robuste pour éviter les abus, notamment la réutilisation de codes ou la manipulation des probabilités de gain.

1.2 Positionnement du service et rôle occupé

Mon alternance ne s'est pas déroulée dans un service informatique traditionnel composé de plusieurs développeurs, chefs de projet et administrateurs système. L'entreprise étant une petite structure, mon rôle a été proche de celui d'un développeur full-stack autonome, en lien direct avec mon maître d'apprentissage. Je devais comprendre le besoin métier, proposer une solution réaliste, développer les interfaces, mettre en place l'API, configurer la base de données et assurer la mise en production.

Cette configuration m'a amené à occuper une position transversale. J'ai travaillé à la fois sur la partie fonctionnelle, en traduisant les besoins du restaurant en parcours utilisateur, et sur la partie technique, en choisissant une architecture adaptée au projet. Les principaux interlocuteurs étaient mon tuteur, les personnes susceptibles d'utiliser l'interface d'administration et les utilisateurs finaux représentés par les clients du restaurant.

Le service concerné peut être assimilé à une activité de digitalisation interne et de marketing numérique. L'objectif n'était pas de vendre un logiciel à grande échelle, mais de mettre en place un outil numérique opérationnel pour un besoin précis. Cela a influencé mes choix : privilégier la simplicité, éviter les dépendances inutiles, construire une interface mobile-first, sécuriser les actions critiques et faciliter la maintenance.

Mon positionnement au sein de cette mission m'a permis d'appliquer de nombreuses notions vues en formation : architecture d'une application web, API REST, authentification, base de données relationnelle, conteneurisation, gestion de projet agile et qualité de code. Il m'a également obligé à développer mon autonomie et ma capacité à expliquer des choix techniques à des interlocuteurs non techniques.

Partie 2 - Les missions réalisées

2.1 Mission 1 : cadrage, UX et site client Matisse Food

La première mission a consisté à cadrer le besoin puis à concevoir la partie visible par les clients. L'enjeu était de créer une expérience suffisamment simple pour être comprise en quelques secondes, car l'utilisateur type est un client du restaurant, souvent sur smartphone, qui vient de recevoir un code en caisse. Il ne faut pas lui demander de créer un compte, de lire une procédure longue ou de naviguer dans une interface complexe.

Le parcours retenu repose sur quatre étapes : commander chez Matisse Food, obtenir un code unique, laisser un avis Google, puis faire tourner une roue de récompense. Ce parcours est présenté dès la page d'accueil afin de limiter les incompréhensions. La page publique affiche également les lots possibles, comme une boisson, une boisson maison, un cookie, des frites cheddar, un burger ou un kebab. Cette présentation sert à créer de l'intérêt tout en gardant un ton adapté à l'univers de la restauration rapide.

J'ai utilisé React et Vite pour développer l'interface. React m'a permis de découper les écrans en pages et composants réutilisables, tandis que Vite a facilité le développement local et la génération d'une version de production. Le routage est assuré par React Router, avec des routes distinctes pour la page d'accueil, la saisie du code, l'étape d'engagement, la roue et le résultat. L'application utilise également Framer Motion pour ajouter des transitions visuelles, ainsi que react-custom-roulette pour l'animation de la roue.

Un point important de cette mission a été le choix d'une interface mobile-first. Les clients utilisent majoritairement leur téléphone dans le contexte du restaurant. Les éléments d'interface ont donc été conçus pour être lisibles, avec des boutons larges, des textes courts et une navigation linéaire. La charte visuelle reprend une ambiance sombre et premium, avec un vert bouteille, des accents chauds et des cartes visuelles. Ce choix donne une identité à l'application tout en mettant en avant les actions principales.

La page de saisie du code a été conçue pour réduire les erreurs. Le champ transforme automatiquement le code en majuscules, limite la longueur et affiche les messages d'erreur renvoyés par l'API. Lorsqu'un code est valide, l'identifiant technique du code est stocké temporairement en session afin de poursuivre le parcours sans exposer d'informations sensibles. L'utilisateur est ensuite redirigé vers une page d'engagement qui ouvre le lien Google Maps du restaurant et déclenche un minuteur avant de permettre l'accès à la roue.

Cette étape d'engagement a demandé un compromis. Il fallait encourager l'utilisateur à laisser un avis sans rendre l'expérience confuse. La solution retenue est une page claire indiquant que l'avis soutient le restaurant, suivie d'un délai court avant le lancement du jeu. Le lien Google Maps est paramétré côté backend via les settings publics, ce qui évite de devoir modifier le code si l'URL change.

La principale difficulté de cette mission a été de garder un parcours fluide tout en respectant les contraintes du jeu. Si le client ferme la page, revient en arrière ou recharge l'application, il faut éviter les incohérences. Pour cela, certaines informations temporaires sont stockées en sessionStorage. Ce stockage est volontairement limité à la session de navigation : il sert à conserver l'état du parcours, mais ne remplace pas la sécurité côté serveur. Le serveur reste la source de vérité pour savoir si un code est valide, joué ou déjà réclamé.

Cette mission m'a permis de travailler sur l'ergonomie, la hiérarchisation visuelle et l'adaptation d'une interface à un contexte d'usage réel. Elle m'a aussi montré que la réussite d'un projet ne dépend pas seulement de la technologie employée. Pour un utilisateur final, ce qui compte est d'abord la compréhension immédiate du parcours, la rapidité d'exécution et la confiance dans le résultat affiché.

2.2 Mission 2 : API, base de données et logique de jeu sécurisée

La deuxième mission a porté sur la création du backend et de la base de données. Cette partie est centrale car elle garantit le bon fonctionnement du jeu. Les règles ne doivent pas être contrôlées uniquement par le navigateur : un utilisateur pourrait modifier le code frontend, rejouer une requête ou tenter d'obtenir un lot sans code valide. La logique sensible devait donc rester côté serveur.

Le backend a été développé avec Node.js, Express et Prisma. Express sert à exposer les routes HTTP de l'application, tandis que Prisma assure l'accès à la base PostgreSQL. Le schéma de données comprend notamment les modèles Admin, Code, Prize et Settings. Le modèle Code suit le cycle de vie d'un code : GENERATED, PLAYED, puis REDEEMED. Cette modélisation permet de savoir si un code est encore utilisable, s'il a déjà servi à faire tourner la roue ou si le lot associé a déjà été réclamé.

Les codes sont générés côté administration à partir d'une fonction qui produit des codes courts et lisibles. Les caractères ambigus comme l, O, 0 ou 1 sont évités pour limiter les erreurs en caisse ou lors de la saisie par le client. Chaque code est unique en base de données, et une collision éventuelle est gérée par une nouvelle tentative. L'interface admin permet de générer un nombre configurable de codes, avec une limite pour éviter les créations massives accidentelles.

La gestion des lots repose sur le modèle Prize. Chaque lot possède un nom, une description, une probabilité, un statut actif/inactif et un niveau, par exemple perte, petit lot, lot moyen ou jackpot. Les lots configurés dans la base incluent notamment des boissons, une boisson maison, un cookie, des frites cheddar, un burger ou un kebab. L'intérêt de cette structure est que le restaurant peut faire évoluer les récompenses sans changer le code source de l'application.

L'algorithme de tirage a été implémenté dans un service séparé. Il récupère les lots actifs, calcule les probabilités et effectue une sélection pondérée. Le choix du lot est réalisé côté serveur avec un générateur aléatoire cryptographiquement plus robuste que `Math.random`, afin de réduire les risques de prédiction. Une fois le lot choisi, le code est marqué comme `PLAYED` et le lot est associé au code. Une mise à jour atomique avec verrouillage optimiste permet d'éviter qu'un même code soit joué deux fois si plusieurs requêtes sont envoyées très rapidement.

La sécurité a également été travaillée sur plusieurs niveaux. Les routes d'administration sont protégées par une authentification JWT. Le mot de passe administrateur est haché avec `bcrypt` dans la base de données. Le middleware `Helmet` ajoute des en-têtes de sécurité HTTP, `CORS` est configuré pour gérer l'origine frontend et un `rate limiting` limite le nombre de requêtes. Une limitation plus stricte est appliquée aux routes de jeu afin d'éviter les tentatives répétées de validation ou de tirage.

Les routes publiques sont volontairement limitées. La validation d'un code renvoie seulement l'information nécessaire pour poursuivre le parcours : le code est valide et peut être utilisé. Les probabilités et les détails internes ne sont pas exposés au client. De même, la route publique des lots ne renvoie que les noms et les niveaux nécessaires à l'affichage de la roue. Cette séparation entre données publiques et données admin est essentielle pour éviter les manipulations.

Cette mission m'a confronté à une difficulté importante : rendre le jeu attractif tout en garantissant son intégrité. La tentation aurait été de gérer une partie de la logique côté frontend pour aller plus vite, mais cela aurait fragilisé l'application. En plaçant la sélection du lot, le changement de statut du code et la validation des droits côté serveur, la solution devient plus fiable. Cela correspond à un principe d'architecture important : le frontend affiche et collecte les actions, mais le backend reste responsable des décisions critiques.

2.3 Mission 3 : interface d'administration et suivi opérationnel

La troisième mission a consisté à développer l'interface d'administration disponible sur `/admin`. Cette interface est destinée au personnel ou au responsable du restaurant. Elle devait être simple, car son usage se fait dans un environnement où le temps est limité : service, caisse, clients en attente, vérification rapide d'un lot.

La première fonctionnalité de cette interface est l'authentification. L'administrateur se connecte avec un identifiant et un mot de passe. En cas de succès, l'API renvoie un token JWT stocké côté navigateur. Les requêtes admin ajoutent ensuite automatiquement ce token dans l'en-tête `Authorization`. Si le token est absent, invalide ou expiré, l'utilisateur est redirigé vers la page de connexion. Cette mécanique permet de protéger les actions sensibles comme la génération de codes ou la modification des probabilités.

Le tableau de bord affiche une vue synthétique de l'activité : nombre total de codes, codes disponibles, codes joués, codes réclamés, taux de conversion et taux de réclamation. Ces

indicateurs donnent au restaurant une lecture rapide de l'efficacité de l'opération. Par exemple, un taux de conversion faible peut indiquer que les clients ne vont pas jusqu'au bout du parcours, tandis qu'un taux de réclamation élevé peut montrer que les lots sont suffisamment attractifs.

La gestion des codes est une fonctionnalité opérationnelle essentielle. Depuis l'admin, il est possible de générer un nombre de codes, de consulter la liste des codes, de filtrer par statut et de voir les lots associés. Cette page sert de suivi interne : elle montre quels codes sont encore disponibles, lesquels ont été joués et lesquels ont déjà été réclamés. La pagination évite de charger trop de données d'un coup lorsque le nombre de codes augmente.

La gestion des lots permet de modifier les récompenses et leurs probabilités. Chaque lot peut être renommé, décrit, activé ou désactivé, et sa probabilité peut être ajustée. L'interface affiche le total des probabilités afin d'alerter si la configuration ne correspond pas à 100 %. Cette fonctionnalité est importante car elle donne de l'autonomie au restaurant : il peut adapter la campagne selon ses stocks, ses marges ou ses priorités commerciales.

La fonctionnalité de réclamation de lot a été conçue pour la caisse. Le personnel saisit le code présenté par le client, l'application retrouve le statut et le lot associé, puis permet de valider la réclamation si les conditions sont remplies. Plusieurs cas sont gérés : code introuvable, code pas encore joué, lot déjà réclamé, perte, ou lot valable. Cette distinction réduit les erreurs et limite les conflits potentiels avec les clients.

Les statistiques et la réclamation ont demandé une attention particulière car elles mélangent logique métier et expérience utilisateur. Il ne suffit pas d'afficher des données : il faut afficher les bonnes données au bon moment, avec un vocabulaire compréhensible. Les statuts techniques comme GENERATED, PLAYED ou REDEEMED sont utiles pour le backend, mais doivent rester lisibles dans l'interface grâce à des badges, des couleurs et des messages explicites.

Cette mission m'a permis de mieux comprendre le rôle d'une interface admin dans un projet applicatif. Une interface publique peut être très visuelle, mais l'interface admin doit d'abord être efficace, fiable et lisible. Elle constitue le centre de contrôle du projet. Si elle est mal conçue, même une bonne expérience client peut devenir difficile à gérer au quotidien.

2.4 Mission 4 : déploiement, conteneurisation et mise en production

La quatrième mission a porté sur le déploiement de l'application. Le projet devait être accessible publiquement sur le sous-domaine matissefood.aimassi.dev, avec une interface client et une interface admin. Pour structurer le déploiement, j'ai utilisé Docker et Docker Compose.

Le fichier docker-compose.yml décrit trois services principaux. Le premier est PostgreSQL, qui stocke les codes, les lots, les administrateurs et les paramètres. Le deuxième est le backend Node.js, exposé sur le port interne 3001 et chargé de lancer les migrations Prisma puis le seed au démarrage. Le troisième est le frontend, construit avec Vite puis servi par Nginx dans un conteneur. Le frontend est configuré avec l'URL de l'API publique afin de communiquer avec le backend en production.

L'utilisation de Docker présente plusieurs avantages. Elle isole l'environnement d'exécution, facilite le redémarrage de l'application et rend le déploiement plus reproductible. Au lieu d'installer manuellement toutes les dépendances sur le serveur, chaque service possède son image et ses paramètres. Cela réduit les différences entre environnement local et environnement de production.

Le déploiement comprend également une configuration Nginx côté serveur hôte. Le sous-domaine `matissefood.aimassi.dev` redirige les requêtes vers le conteneur frontend. Le frontend appelle ensuite l'API via l'URL publique configurée. Cette architecture sépare le reverse proxy, l'application frontend, le backend et la base de données. Elle reste simple, mais elle est déjà proche d'une organisation professionnelle.

La mise en production a aussi impliqué la gestion des variables d'environnement. Le backend vérifie au démarrage que les variables essentielles sont présentes, notamment `JWT_SECRET` et `DATABASE_URL`. Ce contrôle permet d'échouer rapidement en cas de mauvaise configuration plutôt que de laisser l'application démarrer dans un état incomplet. Les informations sensibles ne doivent pas être exposées dans le frontend ni dans le rapport.

Les migrations Prisma permettent de créer et maintenir le schéma de base de données. Le seed initialise les données utiles comme l'administrateur, les lots et les paramètres publics. Cette automatisation accélère la mise en place, mais elle doit être manipulée avec prudence en production, notamment pour éviter d'écraser des données existantes. À terme, une amélioration serait de séparer plus strictement les seeds de démonstration et les opérations de migration de production.

Cette mission m'a fait travailler sur la dimension "run" d'un projet, c'est-à-dire ce qui se passe après le développement. Une application ne se limite pas à son code : elle doit être déployable, observable, sauvegardable et maintenable. Même pour un projet de taille limitée, il est important de penser aux logs, aux sauvegardes de base, aux secrets, aux mises à jour et aux procédures de reprise.

2.5 Organisation, outils et méthode de travail

Le travail a été organisé par tickets et par priorités fonctionnelles. Les outils utilisés étaient principalement Notion, Trello, GitHub et Figma. Notion a servi à structurer les idées, les besoins et les décisions. Trello a permis de suivre l'avancement sous forme de tâches. GitHub a été utilisé pour versionner le code source et conserver l'historique du projet. Figma a servi pour la réflexion visuelle et l'organisation de certaines interfaces avant développement.

La méthode de travail s'est rapprochée d'une démarche agile légère. Les besoins étaient décomposés en petites fonctionnalités : page d'accueil, validation de code, étape d'avis, roue, résultat, login admin, génération de codes, gestion des lots, statistiques, réclamation, déploiement. Chaque fonctionnalité pouvait être développée, testée puis ajustée en fonction du retour.

Les tests ont été principalement fonctionnels et manuels. J'ai vérifié les parcours principaux : code valide, code invalide, code déjà joué, tirage d'un lot, affichage du résultat, réclamation en admin, modification de lots et consultation des statistiques. J'ai également vérifié la navigation et l'affichage sur mobile. Pour la suite, l'ajout de tests automatisés serait pertinent, notamment sur l'algorithme de tirage, les statuts des codes et les routes critiques.

La communication avec mon tuteur a été centrée sur le besoin métier. Plutôt que de présenter uniquement des détails techniques, j'ai dû expliquer les conséquences fonctionnelles : pourquoi un code ne doit être utilisable qu'une fois, pourquoi les probabilités ne doivent pas être dans le frontend, pourquoi il faut un compte admin, ou encore pourquoi la validation d'un lot doit être tracée. Cette traduction entre technique et usage fait partie des compétences que j'ai le plus développées.

Partie 3 - Bilan et recul sur les missions

3.1 Regard critique et axes d'amélioration

Avec le recul, le projet Matisse Food répond bien au besoin initial : proposer une expérience client ludique, relier cette expérience à la visibilité Google du restaurant et donner au personnel une interface de gestion. Toutefois, plusieurs axes d'amélioration peuvent être identifiés.

Le premier axe concerne la qualité logicielle. Le projet gagnerait à intégrer des tests unitaires et d'intégration. Les routes de validation, de tirage et de réclamation sont critiques car elles touchent directement à la fiabilité du jeu. Des tests automatisés permettraient de vérifier que les statuts évoluent correctement, qu'un code ne peut pas être réutilisé, que les lots inactifs ne sont pas tirés et que les erreurs sont bien gérées.

Le deuxième axe concerne l'observabilité. En production, il serait utile de disposer de logs plus structurés, d'alertes en cas d'erreur et d'un suivi de disponibilité. Aujourd'hui, les logs Docker permettent déjà d'obtenir des informations, mais un outil plus complet faciliterait la maintenance. Pour un commerce, une indisponibilité pendant une campagne marketing peut avoir un impact direct sur l'expérience client.

Le troisième axe concerne la sécurité et la gestion des accès. L'authentification JWT et le hachage des mots de passe constituent une bonne base. Cependant, l'application pourrait évoluer vers une gestion de plusieurs comptes, avec des rôles différenciés : administrateur, employé de caisse, lecture seule. Cela permettrait de limiter les permissions selon les usages. Une rotation plus formelle des secrets et une politique de mot de passe seraient aussi utiles.

Le quatrième axe concerne les données et la sauvegarde. La base PostgreSQL contient les codes et les historiques de lots. Il faudrait formaliser une stratégie de sauvegarde et de restauration, même simple. Par exemple, une sauvegarde quotidienne automatisée du volume de base de données ou une exportation périodique des codes permettrait de limiter les pertes en cas d'incident serveur.

Le cinquième axe concerne l'expérience client et les aspects légaux. Le site indique qu'il s'agit d'un jeu gratuit sans obligation d'achat et renvoie aux conditions en magasin. Il serait préférable de formaliser davantage les conditions d'utilisation : durée de validité des codes, modalités de retrait des lots, limitation à un lot par code, gestion des litiges, protection des données et mentions légales. Cela renforcerait la confiance et la clarté.

Enfin, l'application pourrait évoluer vers des fonctionnalités marketing plus avancées : QR codes imprimés sur ticket, campagnes limitées dans le temps, export des statistiques, personnalisation des lots selon les périodes, ou encore tableau de bord comparant plusieurs campagnes. Ces évolutions restent secondaires par rapport au besoin initial, mais elles montrent le potentiel du projet.

3.2 Apports à l'entreprise

Mon principal apport à l'entreprise a été la réalisation d'une solution numérique complète et opérationnelle. Le restaurant dispose d'un site public permettant de proposer une expérience de jeu aux clients, ainsi que d'une interface admin pour piloter cette opération. Le projet donne un support concret à une stratégie d'engagement client : transformer une visite au restaurant en interaction numérique et encourager les avis en ligne.

L'application apporte également un gain d'organisation. La génération de codes uniques permet de distribuer les participations de manière contrôlée. Le suivi des statuts évite les doublons et les réutilisations abusives. La validation des lots via l'admin donne au personnel un moyen simple de vérifier si un client peut récupérer sa récompense. Ces éléments réduisent le risque d'erreur par rapport à une gestion entièrement manuelle.

Le projet apporte aussi de la visibilité sur l'activité de la campagne. Les statistiques du dashboard permettent de suivre combien de codes ont été générés, joués et réclamés. Cela donne au restaurant des informations utiles pour ajuster sa communication ou les lots proposés. Même si ces statistiques restent simples, elles introduisent une logique de mesure qui peut aider à prendre de meilleures décisions.

Enfin, l'application donne une image moderne au restaurant. Dans un secteur où les clients sont sensibles à l'expérience, proposer une roue de récompenses accessible par smartphone crée un effet ludique. Cela peut encourager le bouche-à-oreille, renforcer la fidélisation et donner envie aux clients de revenir.

3.3 Apports personnels et projet professionnel

Cette alternance m'a permis de progresser techniquement et professionnellement. Sur le plan technique, j'ai renforcé mes compétences en React, Node.js, Express, Prisma, PostgreSQL et Docker. J'ai travaillé sur une architecture full-stack complète, depuis l'interface utilisateur jusqu'à la base de données et au déploiement. J'ai également mieux compris l'importance de la sécurité applicative, notamment pour les routes publiques et les opérations critiques.

J'ai particulièrement progressé sur la conception d'API REST. Le projet m'a obligé à réfléchir aux responsabilités de chaque route, aux données renvoyées au frontend, aux codes d'erreur et à la validation des entrées. J'ai appris à distinguer ce qui peut être public de ce qui doit rester côté serveur. Cette séparation est essentielle pour construire des applications fiables.

Sur le plan de l'architecture logicielle, le projet m'a montré l'intérêt de découper les responsabilités : composants React pour l'affichage, services API côté frontend, routes Express côté backend, service spécifique pour le tirage, modèle Prisma pour la persistance, Docker pour l'exécution. Même si le projet reste de taille limitée, cette organisation rend le code plus compréhensible et plus évolutif.

Sur le plan humain, j'ai appris à travailler avec un interlocuteur métier qui n'a pas nécessairement le même vocabulaire technique. Il a fallu reformuler, prioriser et expliquer. Cette compétence est très importante pour un développeur, car un projet réussi est rarement uniquement une question de code. Il faut comprendre l'objectif réel, les contraintes terrain et la capacité de l'utilisateur à adopter l'outil.

Cette expérience confirme mon intérêt pour le développement logiciel full-stack et l'architecture d'applications web. Elle m'a donné envie de continuer à construire des solutions robustes, utiles et déployables. Mon projet professionnel s'oriente vers des postes de développeur full-stack ou de développeur backend avec une forte sensibilité à l'architecture, à la qualité de code et à la mise en production.

Conclusion

Cette alternance au sein de la SAS MOUSTAK m'a permis de travailler sur un projet concret, utile et complet. Le développement de l'application Matisse Food a mobilisé des compétences variées : cadrage fonctionnel, UX, React, API Node.js, base PostgreSQL, sécurité, administration, Docker et déploiement. Le projet a abouti à une solution accessible en ligne, avec un parcours client ludique et un back-office permettant au restaurant de gérer l'opération.

Au-delà de la réalisation technique, cette expérience m'a appris à adapter une solution logicielle à un contexte métier. Les contraintes d'un restaurant de proximité ne sont pas celles d'une grande entreprise : il faut aller à l'essentiel, produire une interface simple, limiter la charge d'utilisation et garantir une maintenance raisonnable. Cette réalité m'a aidé à faire des choix plus pragmatiques.

Le bilan est positif. J'ai pu apporter à l'entreprise un outil numérique exploitable, tout en développant mes compétences techniques et professionnelles. Les axes d'amélioration identifiés, comme les tests automatisés, l'observabilité, la gestion des rôles et les sauvegardes, constituent des pistes de progression pour rendre la solution encore plus robuste.

Cette alternance confirme mon choix de poursuivre dans le domaine de l'architecture logicielle et du développement d'applications web. Elle m'a permis de relier les enseignements de ma formation à un besoin réel, et de mieux comprendre les responsabilités d'un développeur dans la

conception, la livraison et la maintenance d'un produit.

Annexes

Annexe 1 - Fiche d'identité de l'entreprise

| Élément | Information |

|---|---|

| Dénomination sociale | SAS MOUSTAK |

| Nom commercial utilisé dans le projet | Matisse Food |

| Forme juridique | SAS |

| Adresse de l'établissement | 10 bis rue Barbes, 94200 Ivry-sur-Seine |

| Numéro SIRET | 92124341600010 |

| Code APE | 5610C - Restauration de type rapide |

| Effectif indiqué dans le contrat | 4 salariés |

| Secteur d'activité | Restauration rapide / commerce de proximité |

| Représentant / tuteur | BOUZEKRI Mihran |

| Fonction indiquée dans la convention | Employé polyvalent |

| Contrat | Contrat d'apprentissage |

| Dates du contrat | Du 02/01/2026 au 09/11/2026 |

| Étudiant | AIDEL Massinissa |

| Formation | Bachelor Architecture des logiciels 3ème année - ESGI Paris |

| Titre visé | Chargé de développement de solutions applicatives ou logicielles - RNCP 39103 |

| CFA / lieu de formation | CFA ANAPIJ - UFA Campus ESGI Paris, 242 rue du Faubourg Saint-Antoine, 75012 Paris |

Annexe 2 - Fiche projet Matisse Food

| Élément | Information |

|---|---|

| Nom du projet | Matisse Food - roue de récompenses |

| URL publique | <https://matissefood.aimassi.dev> |

| URL d'administration | <https://matissefood.aimassi.dev/admin> |

| Dépôt GitHub | <https://github.com/Amassi06/matissefood> |

| Objectif | Créer une expérience de fidélisation et d'engagement client autour d'un jeu de récompenses |

Parcours client	Code unique, avis Google, roue de tirage, affichage du lot
Fonctionnalités admin	Connexion, génération de codes, gestion des lots, statistiques, validation des lots
Frontend	React, Vite, React Router, Framer Motion, react-custom-roulette
Backend	Node.js, Express, Prisma
Base de données	PostgreSQL
Sécurité	JWT, bcrypt, Helmet, CORS, rate limiting, logique de tirage côté serveur
Déploiement	Docker Compose, conteneurs backend/frontend/PostgreSQL, Nginx
Outils projet	Notion, Trello, GitHub, Figma

Annexe 3 - CV mis à jour

Le CV mis à jour est à insérer ici par l'étudiant avant le dépôt final. Il doit mentionner l'alternance chez SAS MOUSTAK / Matisse Food, le poste de développeur full-stack en alternance et les missions principales : conception d'une application React, développement d'une API Node.js, modélisation PostgreSQL/Prisma, interface d'administration, déploiement Docker et suivi par tickets.

Annexe 4 - Clause de confidentialité

La clause de confidentialité signée par l'entreprise et l'étudiant est à joindre au dossier final si elle est exigée par l'école. Elle n'est pas rédigée dans ce document, conformément à la demande de l'étudiant. Attention : les consignes de rendu mentionnent toutefois cette pièce comme obligatoire.